

DPI343

Computer Organization and Assembly Language

Lecture 5

Lectures - prof. em. U. Sukovskis

Labs for local students - assoc. prof. P. Rusakovs

Labs for foreign students - assoc. prof. G. Alksnis

Autumn 2025



RTU
DATORZINĀTNES,
INFORMĀCIJAS
TEHNOLOĢIJAS
UN ENERĢĒTIKAS
FAKULTĀTE

Looping instructions

If you have the following loop structure:

```
mov ecx,10  
cycle:  ::: ;do something  
        dec ecx  
        jnz cycle
```

} *repeat 10 times*

Then you can replace it with:

```
mov     ecx,10  
cycle:  ::: ;do something  
        loop  cycle
```

} *repeat 10 times*

The register ECX is always used by the LOOP instruction as a counter.

When the LOOP instruction is executed by CPU, the register ECX is decremented by 1 and if ECX is not zero then control is transferred to the memory address (label) specified in the operand.

Make sure the initial value of the loop counter in ECX is greater than zero!!!

You may use the **jecxz** instruction to jump over the cycle-forming instructions in the case the calculated loop counter in the register ECX is 0 or negative.

Looping example

Calculate the sum of all positive elements of the given array of words

.data

vect word 1, 2, -3, 4, -5, 6, 7, -8 ; array given

vect	+1	+2	+3	+4	+5	+6	+7	+8	+9	+10	+11	+12	+13	+14	+15
01	00	02	00	FD	FF	04	00	FB	FF	06	00	07	00	F8	FF



mov esi, 0

c: cmp vect[esi], 0

; . . .

add esi, 2 ; to access next element increase index register by 2
 ; (length of each item is 2 bytes)

Looping example (cont.)

Calculate the sum of all positive elements of the given array of words

```
.data
vect    word    1, 2, -3, 4, -5, 6, 7, -8    ; array given
buff    byte    '0000000000', 0            ; output buffer

.code
start proc
    mov     ecx, 8        ; initial counter value
    mov     esi, 0        ; initial index value
    mov     ebx, 0        ; initial sum value

a:      cmp     vect[esi], 0 ; the current element > 0 ?
    jle     next        ; if not, jump to the label next
    mov     ax, vect[esi]
    cwde
    add     ebx, eax      ; add value of the current element to sum
next:
    add     esi, 2        ; increase index value by the size of one item
    loop    a

; Result is in ebx
; Convert unsigned binary value to the string of symbols and show it on screen
; :::::::::::
start endp
end      start
```

Looping instructions

There are two more specific loop instructions with additional condition:

LOOPE / LOOPZ *label*

- Performs the following:
 1. Decrements ECX
 2. If **ECX $\neq$ 0** and **ZF = 1**, then jumps to the *label*
 3. Otherwise goes to the next instruction.

LOOPNE / LOOPNZ *label*

- Performs the following:
 1. Decrements ECX
 2. If **ECX $\neq$ 0** and **ZF = 0**, then jumps to the *label*
 3. Otherwise goes to the next instruction.

Instruction LOOPE (or LOOPNE) itself does not affect the state of the zero flag (ZF), it must be done by other instructions before the looping instruction.

Nested loops

Each LOOP instruction uses the same ECX register for loop counter, so we must ensure that the value of this register is kept individual for each of the nested cycles.

We can save the current loop counter in the stack and restore it just before execution of the LOOP instruction.

```
    :::  
    mov     ecx,5    ; initial value of the counter for external loop  
c1:  push    ecx      ; save current counter value onto stack  
    :::  
    mov     ecx,10   ; initial value of the counter for internal loop  
c2:  push    ecx      ; save current counter value onto stack  
    :::  
    pop     ecx      ; restore counter value for internal loop from the stack  
    loop    c2  
    :::  
    pop     ecx      ; restore counter value for external loop from the stack  
    loop    c1  
    :::
```

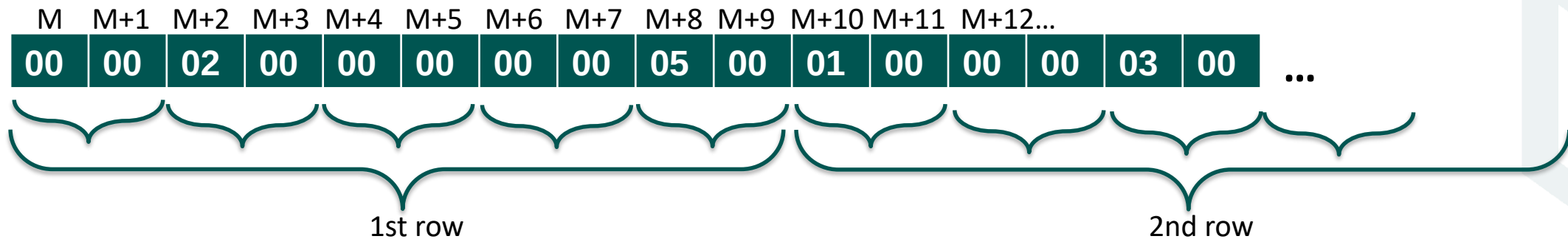
Nested loops. Example

Count number of zero elements in each row of the matrix M (word format).
Store the result for each row in the array R (byte format).

.data

```
M  word  0, 2, 0, 0, 5  
    word  1, 0, 3, 4, 5  
    word  1, 2, 0, 4, 0
```

```
R  byte   3 dup(0)
```



Result should be:

```
R  R+1  R+2  
03  01  02  
...
```

Nested loops. Example (cont.)

Count number of zero elements in each row of the matrix M (word format). Store the result for each row in the array R (byte format).

.data

```
M    word    0, 2, 0, 0, 5
      word    1, 0, 3, 4, 5
      word    1, 2, 0, 4, 0
R    byte    3 dup(0)
```

.code

```
    xor     esi, esi        ; matrix index = 0
    xor     edi, edi        ; result index = 0
    mov     ecx, 3          ; number of rows
```

```
rows:  push  ecx
        xor  al, al          ; number of zeros per row = 0
        mov  ecx, 5          ; number of columns
```

```
cols:  push  ecx
        cmp  M[esi], 0       ; current element of M is 0 ?
        jne  next
        inc  al              ; number of zeros per row + 1
next:   add  esi, 2           ; increase matrix index (size of item is 2 bytes)
        pop  ecx
        loop cols           ; internal loop
```

```
        mov  R[edi], al      ; store number of zeros per row into R
        add  edi, 1          ; increase result index (size of item is 1 byte)
        pop  ecx
        loop rows           ; external loop
```


String instructions

Example: Copy 80 bytes from *field1* to *field2*

```
field1 byte    80 dup(?)
```

```
; :::
```

```
field2 byte    80 dup(?)
```

```
; :::
```

```
; input of the field1 value
```

```
; :::
```

```
;Copy all 80 bytes of the field1 to field2
```

```
    mov     ecx, 80                ; loop counter
```

```
    xor     esi, esi              ; index = 0
```

```
m:  mov     al, field1[esi]        ; move current byte
```

```
    mov     field2[esi], al       ;
```

```
    inc     esi                  ; increment index
```

```
    loop    m
```

Move String instruction

MOVS *destination, source*

MOVSB

MOVSW

MOVSD

Execution of the MOVS instruction:

1. One byte (word or doubleword) is copied from memory address $DS : [ESI]$ to address $ES : [EDI]$
2. If direction flag $DF=0$, then index registers are incremented: $ESI=ESI+n$ and $EDI=EDI+n$
else (i.e. $DF=1$), then decremented $ESI=ESI-n$ and $EDI=EDI-n$
where $n = 1$ for bytes, 2 for words, and 4 for double words

Notes:

1. Operands *destination* and *source* are used by compiler to generate either instruction MOVSB, MOVSW or MOVSD.
2. Registers ESI and EDI must be loaded with values before execution of the string instruction.
Even in the case if the instruction MOVS with operands is used!
3. Before execution the direction flag must be set to 1 or 0 using instruction STD (Set Direction Flag) or CLD (Clear Direction Flag).
4. Repeat prefix REP is often used to repeat execution of the instruction MOVS while counter register $ECX \neq 0$. Prefix REP decrements value of the ECX in the same way as loop instruction.

Move String instruction (cont.)

Example: Copy 80 bytes from *field1* to *field2*

```
field1 byte    80 dup(?)
```

```
; :::
```

```
field2 byte    80 dup(?)
```

```
; :::
```

```
; input of field1 value
```

```
; :::
```

```
mov esi, offset field1    ; ds  already contains data segment value
```

```
mov edi, offset field2    ; es  already contains data segment value
```

```
cld                        ; set DF to 0 to process bytes from left to right
```

```
mov ecx, 80
```

```
rep movsb                ; repeat prefix - ECX=ECX-1 and repeat this single instruction while ECX≠0
```

Compare two text strings

Example:

Compare text in the memory field *psw* with a text stored in the field *sample*

```
psw      byte    8 dup(?)
sample   byte    'qwerty12'
; :::
; input of the psw value
; :::
    xor esi, esi          ; index = 0
    mov ecx, 8            ; loop counter
m:  mov al, psw[esi]       ; copy current byte into AL
    cmp al, sample[esi]   ; compare it with corresponding byte in sample
    jne wrong             ; if current byte does not match go to wrong
    inc esi               ; increment index
    loop m
; :::
wrong:
```

Compare String instruction

CMPS *destination, source*

CMPSB

CMPSW

CMPSD

Execution:

1. Compares memory content at address DS : [ESI] with memory content at ES : [EDI] (one byte, word or double word)
2. If values are not equal, then flags ZF = 0, CF = 1, else ZF = 1, CF = 0
3. If direction flag DF=0, then both index registers are incremented
 $ESI=ESI+n$ and $EDI=EDI+n$
else (i.e. DF=1), then decremented $ESI=ESI-n$ and $EDI=EDI-n$
where $n = 1$ for bytes, 2 for words, and 4 for double words

Notes:

1. Operands *destination* and *source* are used by compiler to generate either instruction CMPSB, CMPSW or CMPSD.
2. Registers ESI and EDI must be loaded with values by program instructions before execution of the string instruction. Even if CMPS instruction with operands is used!
3. Before execution the direction flag must be set to 1 or 0 using instruction STD (Set Direction Flag) or CLD (Clear Direction Flag).
4. Prefix REPNE or REPE is often used to repeat the instruction while condition flags are set, but limited by counter ECX

Compare String instruction (cont.)

Example:

Compare the text entered into string *psw* with the text stored in the *sample*

```
psw      byte    8 dup(?)
sample   byte    'qwerty12'
; :::
; input of the psw value
; :::
    mov     esi, offset psw      ; ESI contains address of the byte to compare
    mov     edi, offset sample   ; EDI contains address of the byte to compare

    cld                      ;from left to right
                          ; (i.e. ESI and EDI will be incremented by cmpsb instruction)
    mov     ecx, 8             ; counter for the prefix REPE

    repe    cmpsb             ; repeat this instruction while ECX≠0 and ZF=0

    jne     wrong             ; if two bytes don't match, jump to the label wrong
; Processing matching strings
; :::
wrong:    ; Processing non matching strings
    dec     esi
    dec     edi
```

Why these two dec instructions could be useful?

Scan String instruction

SCAS *destination*
SCASB
SCASW
SCASD

Execution :

1. Compares $ES : [EDI]$ with AL, AX or EAX and sets appropriate flags
2. If direction flag $DF=0$, then $EDI=EDI+n$
else (i.e. $DF=1$), then $EDI=EDI-n$
where $n = 1$ for AL, 2 for AX, and 4 for EAX

Notes are similar as for MOVS and CMPS.

Very useful to find the first matching element in an array.

Load String instruction

`LODS` *source*

`LODSB`

`LODSW`

`LODSD`

Execution:

1. Copies the contents of memory from the address `DS:[ESI]` into register `AL`, `AX` or `EAX`
2. If flag `DF=0`, then `ESI=ESI+n`
else (i.e. `DF=1`), then `ESI=ESI-n`
where `n = 1` for `AL`, `2` for `AX`, and `4` for `EAX`

Notes are similar as for previous string instructions.

Useful to take and process each element from an array.

Example:

`buffer` byte '12345678'

`:::`

`cld`

`mov esi, offset buffer ; address of the first element`

`mov ecx, 8 ; size of the buffer`

`next: lodsb ; load current byte in AL and increment ESI`

`::: ; process value in the AL`

`loop next`

Store String instruction

STOS *destination*

STOSB

STOSW

STOSD

Execution:

1. Copies the value of AL, AX or EAX register to memory at ES: [EDI]
2. If flag DF=0, then EDI=EDI+n
else (i.e. DF=1), then EDI=EDI-n
where n = 1, for AL; 2, for AX; 4, for EAX

Notes are similar as for previous string instructions.

Useful to fill an array.

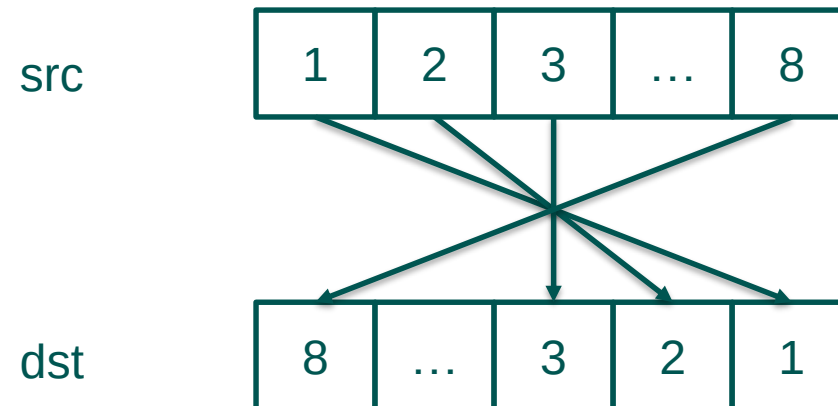
Example:

```
    :::  
buffer byte 80 dup(?)  
    :::  
    mov al, 0FFh  
    mov edi, offset buffer  
    cld  
    mov ecx, 80          ; size of the buffer  
    rep stosb            ; fill all 80 bytes of the buffer  
    :::
```

Example for those who want to try something more

Task: Create reversed array of integers.

E.g., for given source array { 1, 2, 3, 4, 5, 6, 7, 8 } create new destination array { 8, 7, 6, 5, 4, 3, 2, 1 }



Example (continued)

Easy implementation in C++

```
int src[8] = {1,2,3,4,5,6,7,8};
```

```
int dst[8];
```

```
int i, j;
```

```
for (i = 0, j = 7; i < 8; i++, j--) dst[j] = src[i];
```

Example (continued)

```
src  dword  1, 2, 3, -4, 5, -6, 7, -8
dst  dword  8 dup(0)
```

Using general purpose instructions
(very much like we did in C++):

```
    mov esi, 0
    mov edi, 28
    mov ecx, 8

m1: mov eax, src[esi]
    mov dst[edi], eax
    add esi, 4
    sub edi, 4
    loop m1
```

Using string instructions:

```
    mov esi, offset src
    mov edi, offset dst+28
    mov ecx, 8

m2: cld
    lodsd
    std
    stosd
    loop m2
```

Thank You!

